Security Assessment Report

Generated: 2026-05-05 23:54
Report ID: 9f78a82c

Security Assessment Report

1. Executive Summary

A static security review of the provided Python/FastAPI service identified three lowseverity issues, all related to the use of the subprocess module. No critical, high, medium, or informational findings were returned by the automated scans provided. SonarQube reported no issues, and the dependency scan could not be completed because no requirements.txt file was present.

Overall, the application’s risk level is Low based on the available evidence. However, the identified issues are important from a secure coding perspective because they involve command execution patterns and handling of potentially untrusted input. If the application is exposed to usercontrolled paths or extended in the future, these weaknesses could increase the attack surface.

2. Risk Overview

Severity	Count
	:
Critical	0
High	0
Medium	0
Low	3
Informational	0

Risk Interpretation

Critical / High / Medium: No findings were identified at these levels in the provided scan results.

Low: The identified issues indicate insecure or potentially unsafe process execution patterns. While not immediately exploitable based on the current scan output alone, they warrant remediation to reduce the risk of command execution abuse, pathrelated issues, and

maintenancerelated security regressions.

3. Detailed Findings

Low Vulnerabilities

Finding 1 — Use of subprocess Module in SecuritySensitive Context

Severity: Low

CVSS Score (estimated): 3.1

OWASP Category:

A03: Injection

A05: Security Misconfiguration

Affected Component:

./server.py — import of subprocess and scanner execution logic

Description:

The application imports and uses Python’s subprocess module to invoke external commands.

Security tooling flagged this as potentially risky because process execution can become dangerous if command arguments or execution context are influenced by untrusted input.

In this codebase, subprocess.run() is used to call sonarscanner with dynamically derived parameters such as the requestsupplied path.

Although shell invocation is not used, direct process execution still requires strong validation and execution controls.

Impact:

An attacker may be able to influence command behavior indirectly if input validation is weak.

Depending on deployment context, this can lead to unauthorized file access, unintended scanner execution scope, or abuse of local system utilities.

Business impact may include exposure of source code paths, denial of service through repeated scans, or unsafe automation behavior.

Proof of Concept (if possible):

No direct exploit was demonstrated by the scan results.

Risk exists if an attacker can supply a malicious or unexpected path value to influence scanner execution scope.

Recommended Remediation:

Keep using argument arrays rather than shell strings.

Restrict executable paths to absolute, trusted locations.

Validate and normalize all input used in process execution.

Enforce an allowlist of permitted scan directories.

Run the service with least privilege and in a constrained container or sandbox.

Finding 2 — Starting a Process with a Partial Executable Path

Severity: Low

CVSS Score (estimated): 3.7

OWASP Category:

A05: Security Misconfiguration

A03: Injection

Affected Component:

./server.py — subprocess.run(["bandit", ...]) at line 32

Description:

Bandit identified that the application starts a process using a partial executable name ("bandit") rather than a full absolute path.

This can be unsafe because the executed binary is resolved through the system PATH, which may be manipulated in certain environments.

If an attacker can influence the runtime environment, they may cause an unintended executable to run.

Impact:

Potential execution of a malicious or trojanized binary if PATH is compromised.

Could result in arbitrary code execution under the privileges of the application.

Operational impact includes unreliable automation and possible compromise of the host system or container.

Proof of Concept (if possible):

No live exploitation was shown in the scan results.

The issue is environmental and depends on binary resolution through PATH.

Recommended Remediation:

Use absolute paths for all executables, for example /usr/local/bin/bandit.

Validate the runtime environment and restrict PATH.

Prefer container images with pinned and trusted binaries.

Where possible, avoid spawning external processes from application code.

Finding 3 — Subprocess Call with Potentially Untrusted Input

Severity: Low

CVSS Score (estimated): 4.0

OWASP Category:

A03: Injection

A01: Broken Access Control

Affected Component:

./server.py — subprocess.run(["bandit", "r", request.path, ...]) at line 32

Description:

Bandit flagged the subprocess invocation because usercontrolled input (request.path) is passed into the command arguments.

Even though shell=True is not used, untrusted input passed to external tools can still be dangerous if not validated.

The path parameter is accepted from the API request without apparent validation, canonicalization, or restriction to a safe directory.

This could allow a user to direct the scanner to arbitrary filesystem locations.

Impact:

An attacker may trigger scanning of unintended directories or sensitive filesystem locations.

This can expose source code, configuration files, or secrets to the scanning process and logs.

Depending on filesystem permissions and deployment, it may also contribute to denial of service through expensive or recursive scans.

Proof of Concept (if possible):

A malicious user could submit a request with a path pointing to a sensitive or excessively broad directory, causing the scanner to operate outside the intended scope.

Recommended Remediation:

Validate request.path against an allowlist of approved scan directories.

Canonicalize paths and reject relative traversal patterns such as ../.

Enforce that the supplied path stays within a predefined workspace root.

Add authorization controls to ensure only trusted users can initiate scans.

Log and audit scan requests, but avoid logging sensitive path contents unnecessarily.

4. Recommendations (Global)

1. Implement strict input validation

Validate the path field with an allowlist and path canonicalization.

Reject symbolic links, traversal sequences, and paths outside the approved workspace.

2. Harden subprocess usage

Use absolute paths for all external binaries.

Avoid invoking external tools from the web application where possible.

If process execution is required, isolate it in a dedicated worker with reduced privileges.

3. Apply least privilege

Run the service as a nonroot user.

Restrict filesystem access to only required directories.

Use container security controls such as readonly filesystems and dropped capabilities.

4. Improve dependency management

Add a requirements.txt or equivalent dependency manifest.

Pin package versions and regularly update dependencies.

Integrate continuous dependency scanning to detect known vulnerabilities.

5. Strengthen authentication and authorization

Restrict scan execution to authenticated and authorized users.

Add rolebased access control for scan submission and result retrieval.

6. Enhance operational resilience

Add rate limiting to prevent scan abuse.

Implement request size and path length limits.

Replace fixed sleepbased waiting with eventdriven or statuspolling logic where possible.

7. Adopt secure coding practices

Treat all user input as untrusted.

Avoid logging sensitive values such as tokens, secret paths, or full command lines.

Add exception handling that does not leak internal system details.

5. Conclusion

The provided scan results indicate a generally lowrisk security posture with three lowseverity findings concentrated around subprocess execution and input handling. No critical, high, or medium issues were identified in the supplied analysis.

While the current findings are not immediately severe, they should be addressed promptly because they relate to patterns that can lead to command execution abuse or unintended file system access if the application is expanded or misconfigured. Remediating these issues will improve the service’s resilience, reduce operational risk, and align the implementation more closely with secure coding and OWASP best practices.